Guía de secuencia — Laboratorio TRITON

Del clonado del repositorio al taller listo para impartirse

Taller: Análisis forense OT con asistencia de IA **Evento:** CyberForce Summit 2026 · Track TECH-COM **Duración del taller:** 2 h

Modelo: local, sin cuentas ni internet **Cadena verificada:** septiembre 2026, Ubuntu 24.04

> **La regla que decide si esto funciona**
>
> Todo el trabajo se hace con un **usuario sin privilegios**. Nunca root, nunca `sudo`. Wireshark deshabilita los scripts Lua cuando corre como superusuario, el disector de TriStation no carga, y **no hay mensaje de error**: simplemente verás `UDP` en lugar de `TRISTATION` y el taller se queda sin sustancia. Es la falla número uno y cuesta media hora encontrarla en vivo.

0 Antes de empezar

10 min

Reúne esto antes de tocar la terminal. Nada de lo que sigue funciona sin ello.

- [ ] Hipervisor instalado y con espacio (VirtualBox, VMware, UTM o Proxmox)
- [ ] ISO de **Ubuntu Server 24.04 LTS** descargada — es la versión con la que se verificó todo
- [ ] Acceso al repositorio privado `cyberforcesummit2026_workshop`
- [ ] Credenciales de GitHub funcionando (token o SSH) — pruébalas antes
- [ ] Una máquina anfitriona con **holgura para asignarle 8 GB de RAM** a la VM
- [ ] Red decente para la descarga inicial: **son ~5 GB**, la mayoría pesos del modelo
- [ ] Al menos **2 horas libres** para llegar hasta el ensayo sin interrupciones

> **Lo que ya no necesitas**
>
> Ni cuentas de proveedores de IA, ni tokens, ni que el venue deje pasar ningún dominio. Con el modelo en local, la única red que hace falta es la de la preparación.

1 Construir la VM

25 min

Recurso	Mínimo	Recomendado	Por qué
vCPU	4	8	La inferencia en CPU escala con núcleos
RAM	8 GB	16 GB	El modelo ocupa ~4 GB residentes
Disco	30 GB	40 GB	Pesos, Ollama, sistema y evidencia
Red	NAT con internet		Sólo durante la preparación

Crear el usuario del laboratorio

No es cosmético. Es requisito funcional, por lo que dice el recuadro rojo de arriba.

```
sudo adduser analista
sudo usermod -aG sudo analista
su - analista
```

A partir de aquí, **todo se hace como `analista`**.

2 Clonar y provisionar

15 min

```
cd ~
git clone https://github.com/gustavo2107/cyberforcesummit2026_workshop.git taller
cd taller
make bootstrap
```

`make bootstrap` hace todo de corrido y es idempotente — si algo falla a media, lo vuelves a correr sin romper nada. **Descarga unos 5 GB**, así que hazlo con red decente y nunca en el venue:

- Se niega a correr si eres root, y te explica por qué
- Instala `tshark` respondiendo **No** a la pregunta del setuid (aquí sólo se leen archivos PCAP, nunca se captura de la interfaz)
- Descarga la evidencia desde el repositorio de Nozomi e imprime las huellas SHA-256
- Instala el disector en la ruta de plugins que le pregunta a tu propio binario, sin asumirla
- Instala Ollama, descarga el modelo y deja listos el cliente (`ollmcp`) y el servidor (`wireshark-mcp`)
- Corre la verificación al final

Si vas a usar un proveedor en la nube en lugar del modelo local: `SIN_MODELO=1 make bootstrap` .

> **Guarda las huellas SHA-256 que imprime**
> Compártelas con los participantes antes del evento. Es la forma de saber que todos trabajan sobre el mismo archivo — y de paso es una lección de cadena de custodia que sale gratis.

3 Verificar la cadena

2 min

```
./scripts/verify-setup.sh
```

```
[ok] tshark encontrado (TShark (Wireshark) 4.2.2)
[ok] evidencia presente (896 paquetes)
[ok] disector TriStation cargado
[ok] protocolo detectado: TRISTATION
[ok] deteccion TRITON en 2 tramas
[ok] ollama encontrado
[ok] servicio de ollama respondiendo en localhost:11434
[ok] modelo afinado 'triton-analista' disponible

LISTO — el taller es viable en esta maquina.
```

No avances sin todas las líneas `[ok]` . Todo lo que sigue depende de que el disector esté cargado. Si alguna falla, ve a la tarjeta de fallas al final de esta guía.

Comprobación rápida de que la evidencia es la correcta

```
make stats     # 896 paquetes, 435 s, 192.168.1.88 ↔ 192.168.1.2
make triton    # debe devolver las tramas 75 y 887
```

4 El modelo local y el cliente MCP

30 min

> **Todo el análisis corre dentro de la VM**
> Sin llamadas a proveedores externos, sin cuentas y sin tokens. Durante el taller no hace falta internet, y la evidencia nunca sale de la máquina — que en un taller de forense OT no es un detalle menor.

> **Verificado de punta a punta**
> La cadena completa — `ollmcp` → `wireshark-mcp` → `tshark` del sistema → `TriStation.lua` — se corrió en una VM Ubuntu 24.04 y funciona. **El disector se hereda**, que era la incógnita que decidía si este taller era viable: el modelo ve `TRISTATION`, no `UDP`. Aun así, valídala tú en tu máquina: el hardware cambia.

4.1 Ollama y el modelo

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen3.5:4b     # 3.4 GB, tarda

# afinar: los valores por omisión de Ollama rompen este laboratorio
cd ~/taller                # el Modelfile es ruta relativa al repo
ollama create triton-analista -f setup/Modelfile
ollama list                # debe aparecer triton-analista
```

> **Por qué hay que afinarlo**
> Ollama trae `num_ctx` en **2048 tokens** y `temperature` en **0.8**. Con 2048 la salida de `tshark` desborda el contexto y el modelo pierde la evidencia a media sesión; con 0.8 fabrica justo la confabulación que el taller enseña a detectar. `setup/Modelfile` los lleva a 8192 y 0.1, con seed fijo para que el análisis sea reproducible.

Por qué este modelo y no uno más grande. El laboratorio no le pide al modelo que escriba bien: le pide que **invoque `tshark`, lea el resultado y no invente entre una llamada y otra**. En la evaluación de tool calling de 2026 sobre 40 casos deterministas, Qwen3.5 4B obtuvo **97.5% de acierto** — primer lugar, por encima de modelos cinco veces mayores, en 3.4 GB. Ahí es donde los modelos locales se separan.

4.2 El cliente y el servidor MCP

```
sudo apt install -y pipx     # NO viene instalado por omisión
pipx install ollmcp          # el cliente
pipx install wireshark-mcp   # el servidor que expone tshark
pipx ensurepath
exec $SHELL                  # sin esto: "ollmcp: command not found"

cd ~/taller
ollmcp -m triton-analista -j setup/mcp-config.minimo.json
```

> **Dos trampas en una sola línea**
> **El flag es `--servers-json` (`-j`)**, no `--mcp-server`: ése existe, pero espera la ruta a un script `.py` o `.js`, y con el nombre de un binario responde `Server script not found`.
> **Y el servidor es `wireshark-mcp`, no `mcp-wireshark`.** El segundo instala limpio y no arranca: revienta con `AttributeError: 'Server' object has no attribute 'list_tools'`, por incompatibilidad con el SDK actual de MCP. No hay flag que lo arregle.

> **Con pipx, no con pip**
> Desde Ubuntu 24.04 rige PEP 668: `pip install --user` falla con `externally-managed-environment` porque el Python del sistema lo gestiona apt. `pipx` es además la vía correcta para una aplicación de línea de comandos — entorno aislado y binario en el PATH.

4.3 Fijar los parámetros dentro del cliente

Este paso no es opcional, y es el que más fácil se salta. `ollmcp` lleva su **propia** configuración de modelo. Aunque hayas construido `triton-analista` con el `Modelfile`, al abrir el panel de parámetros puedes encontrarte esto:

```
/mc

num_ctx        -
temperature    -
seed           -
```

Un guión no quiere decir 8192, ni 0.1: quiere decir que **el cliente no está enviando ese parámetro**. Con `num_ctx` sin fijar vuelves al problema de raíz — el modelo se queda sin contexto a media línea de tiempo y, como no sabe decir "se me olvidó", rellena. Fíjalos ahí mismo y guárdalos:

Parámetro	Valor
`num_ctx`	`8192`
`temperature`	`0.1`
`seed`	`42`
`/sc`	Guarda en `~/.config/ollmcp/config.json`

> **Ese archivo tiene que ir dentro de la OVA**
> Si exportas la imagen sin `~/.config/ollmcp/config.json`, cada participante arranca con los parámetros en `-` y el laboratorio se comporta distinto en cada máquina — que es justo lo que la snapshot existe para evitar. Confírmalo con `cat` antes del paso 5.

4.4 La pregunta de control

El mismo día del taller, antes de que llegue nadie:

```
Corre: tshark -r malware_exec.pcap -c 1
y dime qué protocolo reporta.
```

> **Debe responder TRISTATION — y debes ver la llamada a la herramienta**
> Hay dos formas de fallar y se arreglan distinto. Distínguelas antes de seguir.

Qué ves	Qué pasa	Qué hacer
Responde `UDP`	El disector no cargó: el proceso corre como root o sin el plugin	Tarjeta de fallas, primera fila
Responde algo plausible **sin llamar la herramienta**	El modelo está inventando	**Cambia de modelo. No sigas.**

El segundo caso es el grave: es exactamente el fallo que el taller entero enseña a detectar. Si el modelo lo hace en la validación, lo va a hacer toda la sesión.

Cuando responde bien, la prueba dura de que el disector llegó hasta el modelo es el campo `frame.protocols`:

```
"frame.protocols": "eth:ethertype:ip:udp:tristation"
```

Si termina en `udp`, la cadena está rota aunque el modelo suene convincente.

4.5 Lo que vas a ver, y no significa lo que parece

Tres cosas de la *herramienta* —no del caso— que salen en cuanto empiezas a consultar. Ninguna es un problema; todas confunden si te agarran en frío:

- Las herramientas se listan con `/tools`, y **su numeración cambia entre versiones del servidor**. No memorices números ni los pongas en el material del participante: lo que se dicta en voz alta es el nombre.
- Pedir la lista de paquetes con `limit: 1` devuelve **cabeceras, no contenido**. Para ver los campos que decodifica el disector, la vía sigue siendo `tshark -r <archivo> -c 1 -V`.
- El resumen de *expert info* da **frecuencias, no números de trama**. Sirve para saber *qué* aparece y *cuántas veces*; para saber *dónde*, hace falta otra consulta.

4.6 Cuánto tarda

En la VM de referencia, cada consulta —la ida y vuelta completa: el modelo decide, llama la herramienta, lee el resultado y responde— tardó alrededor de **tres minutos** sobre CPU.

> **Mídelo en tu máquina y dilo en voz alta**
> Ese número depende por entero del equipo: núcleos, RAM y si hay GPU. Sin el aviso, alguien da por muerto el cliente a los veinte segundos y lo reinicia a media consulta, perdiendo todo el contexto que llevaba anclado. El corolario que también conviene decir: **cada consulta cuesta**, así que vale más pensar la pregunta que dispararla.

4.7 Si el modelo local no da el ancho

No lo fuerces. La ruta con un proveedor en la nube sigue siendo válida (`SIN_MODELO=1 make bootstrap`). Pierdes la independencia de la red del venue y ganas capacidad. **Decídelo en tu ensayo, no en el salón.**

5 Snapshot

5 min

Con la verificación en verde y el MCP respondiendo TRISTATION, limpia y congela:

```
cat ~/.config/ollmcp/config.json    # PRIMERO: confirma que existe
history -c
sudo apt clean
sudo rm -rf /tmp/*
```

Y toma un snapshot del hipervisor llamado **`baseline-verificada`**.

Es tu red de seguridad: si un participante destruye su entorno a media sesión, reviertes en treinta segundos en lugar de perderlo. **Tómala después de verificar, nunca antes.**

6 Ensayar

2 h

El paso que todo mundo se salta y del que todo mundo se arrepiente. Corre el workbook completo tú mismo, con cronómetro, como si fueras participante — sin abrir la rúbrica.

- [ ] ¿Alcanzan los 30 minutos del Bloque 3?
- [ ] ¿Llegaste solo a la trama 892, o necesitaste la pista?
- [ ] ¿Encontraste el `keyState`, o te lo dijo el modelo de memoria?
- [ ] ¿Cuántas afirmaciones de tu asistente no pudiste anclar a una trama?
- [ ] ¿Cuánto tardó cada consulta **en esta máquina**? Ése es el número que dices en voz alta.

Después contrasta contra `docs/03-rubrica.md`. Lo que se te dificultó a ti se le va a dificultar a la sala, y ahí es donde ajustas las pistas.

Lo que te va a pasar en el ensayo — y es la mejor parte del taller

En la verificación de este laboratorio, al pedirle el resumen de *expert info*, la herramienta devolvió esto:

```
Errors (3)
  1  Undecoded  TRISTATION  Lua Error: TriStation.lua:611: negative length in tvb range
  2  Malformed  TRISTATION  TRITON malware detected!

Notes (2)
  2  Protocol   Ethertype   Didn't find padding of zeros...
```

El modelo produjo, a partir de ahí, **un hallazgo crítico correcto envuelto en dos sobreinterpretaciones perfectamente plausibles**:

Lo que leyó	Lo que concluyó	Qué es en realidad
`TRITON malware detected!`	Dos tramas con firma de TRITON	**Correcto.** Es el hallazgo, y viene del disector
`Lua Error: TriStation.lua:611`	"Paquetes corruptos en la captura"	Un **bug del disector**, de 2018. Dispara una vez, siempre la misma trama
`Didn't find padding of zeros`	Posible "manipulación del paquete"	Aviso normal de Wireshark sobre el relleno de la trama Ethernet

> **Las reglas de anclaje no atrapan esto**
> El modelo etiquetó las tres afirmaciones como `[EVIDENCIA]` — y en su lógica lo eran: salieron de ejecutar un comando y citan literalmente lo que la herramienta imprimió. Lo que falta es el paso siguiente, y hay que enseñarlo explícito: **¿esto es una propiedad del tráfico, o del decodificador que lo está leyendo?** Ésa es la pregunta que cierra el taller. Está desarrollada en `docs/03-rubrica.md`, y ya es criterio de calificación con peso alto.

Puedes dejar que le pase a algún equipo y recogerlo en la revelación, o proyectar el bloque de arriba y pedirle a la sala que diga qué concluiría. Funciona igual de bien, y no depende de que alguien pregunte lo correcto.

Graba el video de respaldo

Con el modelo en local ya no dependes de la red del venue, así que la falla más común desapareció. Pero el video sigue sin ser opcional, por otra razón: **un modelo pequeño puede portarse mal en vivo**. Si eso pasa, el video te deja seguir dando el taller como demo guiada en lugar de perderlo.

7 Semana del taller —

- [ ] Publicar el brief y la guía de la VM **48 horas antes** — que nadie descargue nada en el salón
- [ ] Comunicar las huellas SHA-256 a los participantes
- [ ] Exportar la OVA y copiarla a **dos memorias USB** (no la distribuyas por el WiFi del venue)
- [ ] Imprimir el workbook — hay quien trabaja mejor en papel
- [ ] Confirmar que las laptops de los participantes pueden asignar **8 GB** a la VM
- [ ] Validar la pregunta de control **el mismo día** — el modelo debe responder TRISTATION
- [ ] Revisar `/mc` el mismo día: ningún parámetro en `-`
- [ ] Tener a la mano el tiempo por consulta medido, para anunciarlo al abrir
- [ ] Preparar **dos mesas de rescate** con el entorno ya corriendo
- [ ] Probar la VM en una máquina **que no sea la tuya**

8 Tarjeta de fallas

Síntoma	Causa	Solución
Protocolo aparece como `UDP` y payload `data`	Corriendo como root: Wireshark deshabilita Lua para superusuario	Usar el usuario sin privilegios. Nunca `sudo`.
`tshark -G protocols | grep tristation` no devuelve nada	El disector no cargó	Confirmar la ruta con `tshark -G folders`, o usar `-X lua_script:`
Las huellas SHA-256 no coinciden	Alguien trabaja sobre otro archivo	Volver a correr `make evidence` y comparar antes de seguir
El asistente responde `UDP` a la pregunta de validación	El proceso del MCP corre como root o con otro usuario	Revisar bajo qué usuario arranca el MCP
El asistente responde sin llamar la herramienta	El modelo está inventando en lugar de consultar	Cambiar de modelo. No es recuperable con prompts.
`pipx: command not found`	No viene instalado en Ubuntu por omisión	`sudo apt install -y pipx`
`ollmcp: command not found` tras instalarlo	Quedó en `~/.local/bin`, fuera del PATH de la sesión	`pipx ensurepath && exec $SHELL`
`pip`: `externally-managed-environment`	PEP 668 en Ubuntu 24.04	Usar `pipx` para aplicaciones, `apt` para librerías
`no Modelfile or safetensors files found`	Ruta relativa: no estás dentro del repositorio	`cd ~/taller` antes de `ollama create`
`ollama` no responde o el modelo no carga	Servicio caído, o RAM insuficiente en la VM	`ollama ps`. Subir la RAM a 8 GB o más.
Cada consulta tarda ~3 minutos	**Es normal en CPU.** No es una falla	Anúncialo al abrir la sesión. Más vCPU y RAM lo bajan; una GPU, mucho más.
`Server script not found`	Usaste `--mcp-server`: espera un `.py` o `.js`	`-j setup/mcp-config.minimo.json`
`AttributeError: 'Server' object has no attribute 'list_tools'`	Es `mcp-wireshark`: incompatible con el SDK actual de MCP	`pipx install wireshark-mcp` y cambiar el `command` del JSON
En `/mc` los parámetros aparecen como `-`	El cliente no los está enviando: el `Modelfile` no basta	Fijarlos en `/mc` y guardar con `/sc`
El modelo reporta "paquetes corruptos"	Leyó el bug de Lua del disector como propiedad del tráfico	No se arregla: **es material del taller.** Ver la sección 6
El asistente inventa IOCs o dominios	Falta la regla de anclaje, o el disector no está cargado	Pegar `prompts/00-reglas-de-anclaje.md` antes de cualquier pregunta
El conteo de paquetes no es 896	PCAP incorrecto o truncado	`make clean && make evidence`

9 Referencia rápida

Comando	Qué hace
`make bootstrap`	Provisiona la VM de cero a verificada
`make evidence`	Descarga la evidencia desde el origen
`make verify`	Valida la cadena completa
`make stats`	Resumen de la captura
`make triton`	Tramas donde el disector detecta TRITON
`make clean`	Borra la evidencia descargada
`make hooks`	Activa el hook de pre-commit
`make check-docs`	Verifica que no falte documentación
`ollama list`	Modelos descargados
`ollama ps`	Si el modelo está cargado en memoria
`ollmcp -m triton-analista -j setup/mcp-config.minimo.json`	Arranca el cliente con el servidor MCP

Dentro del cliente

Comando	Qué hace
`/mc`	Panel de parámetros del modelo. **Ninguno debe decir `-`**
`/sc`	Guarda la configuración en `~/.config/ollmcp/config.json`
`/tools`	Herramientas disponibles. La numeración cambia entre versiones

Los números que debes reconocer de memoria

Dato	Valor
Paquetes / duración	896 / 435.39 s
Hosts	192.168.1.88 (estación) ↔ 192.168.1.2 (Triconex), UDP 1502
Ruido de fondo	676 tramas (75%) de *Get CP status*
Detecciones de TRITON	Tramas 75 y 887
El detalle que casi nadie ve	Trama 891 *Run program* → trama 892 **Command rejected**
El hallazgo	`keyState = 0x01 [Program]` constante en las 338 respuestas
Lo que **no** es del caso	El `Lua Error` de `TriStation.lua:611` y el aviso de *padding* de Ethernet: artefactos de la herramienta
La prueba de que el disector llegó al modelo	`frame.protocols` termina en `:tristation`